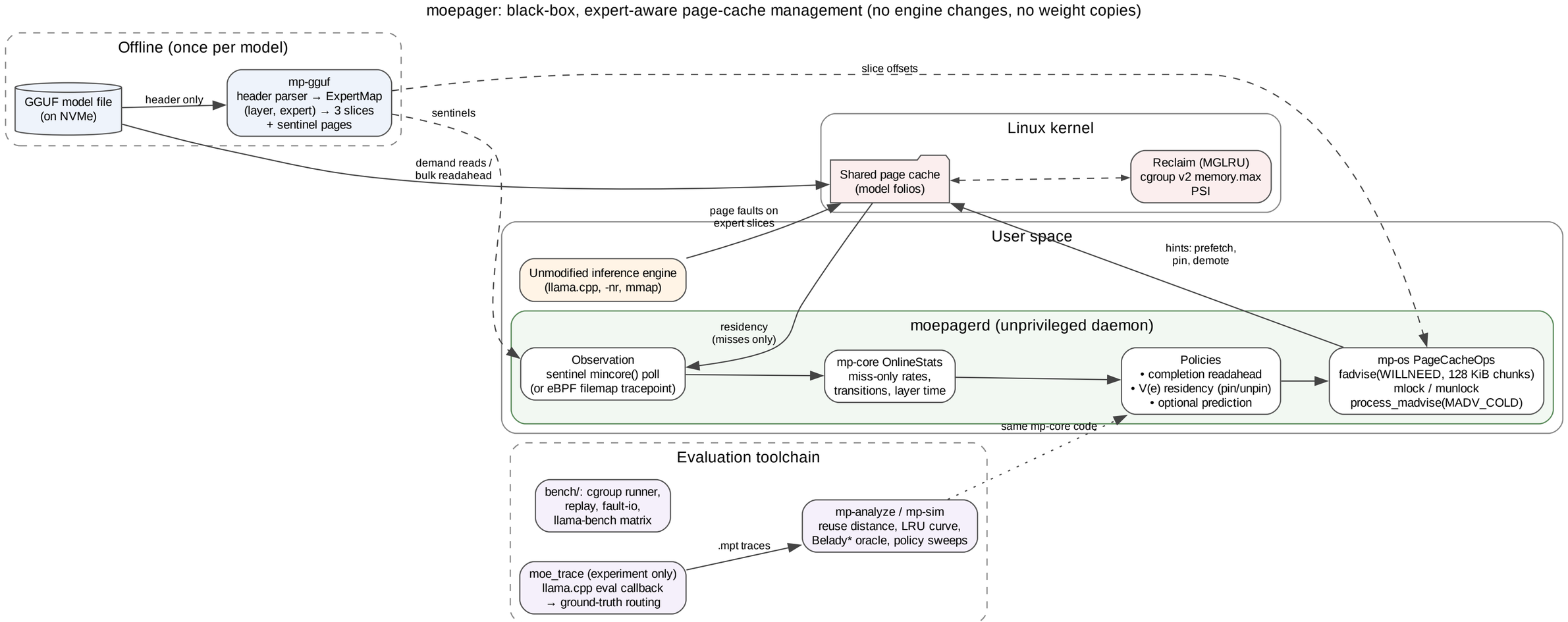
moepager: black-box, expert-aware page-cache management (no engine changes, no weight copies)
Offline (once per model)
GGUF model file
(on NVMe)
header only
mp-gguf
header parser → ExpertMap
(layer, expert) → 3 slices
+ sentinel pages
slice offsets
sentinels
demand reads /
bulk readahead
Linux kernel
Shared page cache
(model folios)
Reclaim (MGLRU)
cgroup v2 memory.max
PSI
page faults on
expert slices
User space
hints: prefetch,
pin, demote
Unmodified inference engine
(llama.cpp, -nr, mmap)
moepagerd (unprivileged daemon)
residency
(misses only)
Observation
sentinel mincore() poll
(or eBPF filemap tracepoint)
mp-core OnlineStats
miss-only rates,
transitions, layer time
Policies
• completion readahead
• V(e) residency (pin/unpin)
• optional prediction
mp-os PageCacheOps
fadvise(WILLNEED, 128 KiB chunks)
mlock / munlock
process_madvise(MADV_COLD)
same mp-core code
Evaluation toolchain
bench/: cgroup runner,
replay, fault-io,
llama-bench matrix
mp-analyze / mp-sim
reuse distance, LRU curve,
Belady* oracle, policy sweeps
.mpt traces
moe_trace (experiment only)
llama.cpp eval callback
→ ground-truth routing